

UNIT 2

COMPUTATIONAL THINKING AND ALGORITHM

10TH COMPUTER SCIENCE FEDERAL BOARD

AWINASH GOSWAMI

AGENDA

- Introduction
 - Computing Problems
 - Algorithm And Characteristics
 - Algorithm Applications
 - Boolean Logic
 - Standard Algorithms
 - Abstraction
 - Creating Generalized Solutions
 - Modular Design
 - Algorithm Dry Run
 - Errors
-

INTRODUCTION

- Computational Thinking (CT) is a way of solving problems using ideas from computer science. It helps break big problems into smaller parts and create step-by-step solutions that a computer or person can follow.
-

COMPUTING PROBLEMS

- A computing problem in computer science is one that is solved step-by-step through computation. It can include any type of calculation — arithmetical or logical. These problems usually have a well-defined input and an output that must satisfy desired properties.
 - **Examples:**
 - Finding the largest number in a list.
 - Checking if a number is even or odd.
 - Simulating a scientific phenomenon.
 - **Computing problem types:**
 - **Decision Problem** - In decision problems normally yes-or-no response is found in a choice. E.g., given a number n , "is n even?"
 - **Search Problem** - In search problems, the solution is made up by having a set of values that satisfy a certain criteria. E.g. : we might wish to calculate a route on a map from one site to another (e.g. from Islamabad to Lahore)
-

ALGORITHMS AND THEIR CHARACTERISTICS

- Algorithm is a well-defined, step-by-step procedure or set of rules for solving a specific problem or performing a task. Computer science relies heavily on algorithms for processing data, automating reasoning, and doing calculations. They can be expressed through computer languages, flowcharts, pseudocode, or natural language.
 - **Properties of Algorithms:**
 - **Input** – One or more externally supplied values.
 - **Output** – At least one result from the calculations.
 - **Definiteness** – Each step must be clearly and accurately stated.
 - **Finiteness** – Must end after a finite number of steps.
 - **Effectiveness** – Operations must be simple enough to do with paper and pencil.
 - **Generality** – Should solve a group of problems, not just one.
-

ALGORITHM APPLICATIONS

- Algorithms can be applied to solve various problems, such as:
 - **Sorting and Searching** – Organizing and retrieving data efficiently using algorithms like **quick sort**, **merge sort**, and **binary search**.
 - **Graph Algorithms** – Finding the shortest paths in navigation apps using the **Dijkstra algorithm**.
 - **Cryptography** – Encrypting and decrypting data for **privacy and security** in online transactions.
 - **Machine Learning** – Learning from data and making predictions using algorithms like **decision trees** and **neural networks**.
 - **Image Processing** – Enhancing, compressing, or analyzing images, with critical roles in **medical imaging**, **computer vision**, and **multimedia applications**.
-

BOOLEAN LOGIC

- In Boolean, two statements or expressions are evaluated in a way that can only have one of two potential values as a result —either true or false. Boolean logic uses logical operator like AND, OR, NOT, etc.
- Following are few of the most widely used operators with truth table.

A	B	A AND B	A OR B	NOT A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

STANDARD ALGORITHMS

- Standard Algorithms are fundamental methods used to solve common problems in computational thinking.
 - Sorting Algorithms arrange data in a specific order (ascending or descending):
 - **Bubble Sort** – Repeatedly compares adjacent elements and swaps them if they are in the wrong order.
 - **Selection Sort** – Divides the list into sorted and unsorted parts, repeatedly selecting the smallest/largest element from the unsorted part.
 - **Merge Sort** – Divides the list into halves, recursively sorts each half, then merges them.
 - **Insertion Sort** – Builds the sorted array one element at a time, shifting larger elements to the right.
-

STANDARD ALGORITHMS

- Searching Algorithms are used to find a target element in a list or array:
 - **Linear Search** – Sequentially checks each element in a list until the target is found or the list ends. Works on any list (sorted or unsorted).
 - **Binary Search** – Works on sorted arrays by repeatedly dividing the search interval in half to efficiently find the target's position.
-

STEPS IN ALGORITHM TO SOLVE COMPUTATIONAL PROBLEMS

- Core Concepts of Computational Thinking:
 - **Decomposition** – Breaking complex problems into smaller sub-problems and solving them one at a time. Makes problems simple and understandable.
 - **Example** Instead of thinking about the whole wedding at once, break it into smaller tasks (Book a hall, Order food, Send invitations, Arrange decorations, Hire a photographer)
 - **Pattern Recognition** – Identifying similarities and differences in problems or data to create a general solution. Example - **Weather in Sindh** - Every year, summer is hot and dry, and monsoon brings rain, by recognizing this yearly pattern, farmers decide when to plant crops.
-

STEPS IN ALGORITHM TO SOLVE COMPUTATIONAL PROBLEMS

- **Abstraction** – Focusing on important information and ignoring irrelevant details. Reduces complexity and makes solutions easier to develop.
 - Example **Using Google Maps** - When you use Google Maps to go from Karachi to Hyderabad, you only see **roads and landmarks**, you don't see trees, shops, or small houses — those details are **hidden** because they are not important for navigation.
 - **Example 2: Using a Mobile Phone** - You tap on the **WhatsApp icon** to open the app. You don't need to know the **complex code** behind it, the icon is an **abstraction** of all that code.
 - **Algorithms** – Step-by-step solutions to a problem. Each step defines a rule to follow. Essential for automation and efficiency.
 - Example **Withdrawing Money from ATM** - Insert card. Enter PIN. Select "Withdraw." Enter amount. Collect cash. Take card back.
-

ABSTRACTION

- **Abstraction in Computing:**
 - **Programming Variables** – Represent data generically instead of using constants.
 - **Functions** – Built-in functions are called by name without knowing internal procedures; programmers can also create their own.
 - **Object-Oriented Programming (OOP)** – Real entities are represented as objects, hiding complex implementation details.
-

CREATING GENERALIZED SOLUTIONS

- Problems can be specific, but the best solutions should be generic. A specific solution handles only one situation, while a generic solution can handle similar problems anytime — improving efficiency, scalability, and reusability.
 - **Steps to Create Generalized Solutions:**
 - Identify the Core Problem – Study specific cases and find common essential components.
 - Extract Common Patterns – Find repeated elements and decide if they apply broadly.
 - Develop a Generic Solution – Address core and common elements; use variables instead of constants.
 - Test with Different Scenarios – Try various inputs and modify if needed.
 - **Example 1 – Area of Shapes:**
 - *Specific:* Rectangle area = length × width.
 - *Generalized:* A function that calculates area for different shapes (rectangle, circle, triangle) using their own formulas.
-

MODULAR DESIGN

- Modular Design is a fundamental concept in software engineering and computational thinking. It focuses on breaking down a system into smaller, manageable, interchangeable parts called modules.
 - **Examples:**
 - Programming: Functions or methods are modules that perform specific tasks (e.g., a Python function to calculate the area of a rectangle).
 - OOP: Uses classes and objects to implement modular design.
 - **Steps to Implement Modular Design:**
 - Define Modules – Divide complex problems into sub-problems, and define a module for each core problem.
 - Implement Modules – Build each module according to its algorithmic steps.
 - Integrate Modules – Combine modules into a complete system, ensuring they interact correctly.
 - Test Modules – Test each module separately and the complete system together.
-

ALGORITHM DRY RUN

- Dry Run means manually checking an algorithm by giving it specific inputs. It helps verify correctness, understand behavior, and identify logical errors.
 - **Steps of Dry Run:**
 - Choose an Input – Select specific input values.
 - Simulate Execution – Manually perform each step and track variable states.
 - Observe Output – Note the final output after all steps.
 - Compare with Expected Result – If it matches, the algorithm is correct; if not, review each step to find errors.
-

ALGORITHM DRY RUN CONT....

- **Example – Finding the Largest Number:**

Algorithm:

- Set max = first element of the list.
- Loop through remaining elements.
- Compare each element with max.
- If greater, update max.
- Return max.

- **Input List:** [3, 1, 4, 1, 5, 9, 2, 6]

Dry Run:

- max = 3 → compare 1 (no change) → compare 4 (max = 4) → compare 1 (no) → compare 5 (max = 5) → compare 9 (max = 9) → compare 2 (no) → compare 6 (no)
 - **Output: 9**
-

ALGORITHM DRY RUN

- **Dry Run Example – Finding the Largest Number:**

- **Input List:** [3, 1, 4, 1, 5, 9, 2, 6]

- **Initialization:** max = 3

- **Iteration:**

- Compare 1 with 3 → no change
- Compare 4 with 3 → max = 4
- Compare 1 with 4 → no change
- Compare 5 with 4 → max = 5
- Compare 9 with 5 → max = 9
- Compare 2 with 9 → no change
- Compare 6 with 9 → no change

- **Result:** max = 9 

This matches the visually largest number, proving the algorithm works correctly. Dry runs help **validate algorithm logic before coding**.

ERRORS

- **Errors** – Understanding errors helps develop accurate problem-solving strategies.
 - **Syntax Errors** – Mistakes in the **structure** of an expression or statement, like **grammatical errors** in writing.
 - **Logical Errors** – Mistakes in **reasoning or planning** that lead to incorrect conclusions, like **flawed problem-solving approaches**.
 - **Example – Search Engine Query:**
 - Query: *"Best smartphones for college students under 500"*
Logical error: User meant to exclude used phones but gets them included.
 - Refined: *"Best **new** smartphones for college students under 500."*
Syntax error: Currency not specified.
 - Final refined: *"Best new smartphones for college students under **PKR** 500."*
-